Température et top_k

Comprendre le choix du prochain token dans un Mini-GPT

Ce document présente deux paramètres du décodage autorégressif : la température, qui modifie la distribution des probabilités, et top_k, qui limite le nombre de tokens candidats.

Objectif pédagogique

Comprendre ce qui se passe après que le modèle a calculé les logits, et distinguer l'entraînement du modèle de la stratégie utilisée pour générer du texte.

Mini-GPT • Transformer causal • Décodage autorégressif

1. Du contexte aux logits

À chaque étape, le modèle reçoit une séquence de tokens et calcule un score pour chaque token du vocabulaire. Ces scores sont appelés logits. Un logit élevé indique que le modèle favorise davantage le token correspondant, mais un logit n'est pas lui-même une probabilité.

Exemple fictif de logits pour le prochain token :

Token candidat	Logit
python	4
modèle	3
apprend	2
java	1
voiture	0

Pour transformer les logits en probabilités, on utilise généralement Softmax :

$$P(\text{token } i) = \exp(z_i) / \Sigma_j \exp(z_j)$$

La somme des probabilités vaut 1, soit 100 %. La température agit avant Softmax ; top_k élimine certains candidats avant le calcul final des probabilités.

2. La température

La température, notée T, est un nombre strictement positif qui contrôle à quel point la distribution de probabilités est concentrée ou répartie. On divise chaque logit par T avant d'appliquer Softmax :

$$P(\text{token } i) = \exp(z_i / T) / \Sigma_j \exp(z_j / T)$$

T = 1

Les logits ne changent pas : on obtient la distribution Softmax habituelle.

T inférieure à 1 (par exemple 0,5)

Les différences entre les logits sont amplifiées par la division. Après Softmax, le token le mieux classé reçoit généralement une probabilité plus élevée. La génération devient plus prévisible.

T supérieure à 1 (par exemple 2)

Les différences entre les logits sont atténuées. Les tokens moins bien classés obtiennent relativement plus de probabilité. La génération devient plus variée, mais peut aussi devenir moins cohérente.

À retenir : la température ne réentraîne pas le modèle et ne rend pas ses poids meilleurs. Elle modifie la manière de sélectionner le prochain token à partir des scores déjà calculés.

3. Le filtrage top_k

top_k est un paramètre entier qui indique combien de tokens parmi les mieux classés peuvent rester candidats à la génération. Les logits des autres tokens sont masqués, généralement en les remplaçant par $-\infty$. Après Softmax, leur probabilité devient nulle.

Exemple : top_k = 3

Avec les logits de l'exemple précédent, les trois candidats conservés sont python (4), modèle (3) et apprend (2). Les candidats java et voiture sont éliminés pour cette étape.

Attention : top_k = 3 ne donne pas une probabilité de 1/3 à chaque token. Les trois probabilités dépendent toujours des logits conservés et de la température.

Extrait de code PyTorch

```
values, _ = torch.topk(next_token_logits, k=top_k)
threshold = values[:, -1].unsqueeze(-1)

next_token_logits = next_token_logits.masked_fill(
    next_token_logits < threshold,
    float("-inf"),
)

probabilities = torch.softmax(next_token_logits, dim=-1)
next_id = torch.multinomial(probabilities, num_samples=1)
```

Lecture du code :

1. `torch.topk` récupère les k logits les plus élevés.
2. Le dernier élément de `values` donne le seuil minimal des k candidats retenus.
3. `masked_fill` remplace les logits inférieurs au seuil par $-\infty$.
4. Softmax recalcule les probabilités sur les candidats restants.
5. `torch.multinomial` échantillonne un token selon ces probabilités.

Avec top_k = 1, un seul candidat reste autorisé. Le token choisi est donc toujours celui dont le logit est le plus élevé : c'est la sélection gloutonne (greedy decoding).

4. Comment les deux paramètres se complètent

Paramètre	Rôle	Effet typique
Température	Modifie la concentration des probabilités.	Faible : plus prévisible. Élevée : plus variée.
top_k	Limite le nombre de tokens candidats.	Petit k : choix plus restreint. Grand k : plus de candidats.

5. Exemple dans notre Mini-GPT

```
generated = generate_text(
    model,
    prompt="le machine learning",
    max_new_tokens=40,
    temperature=0.8,
    top_k=20,
)
```

• max_new_tokens = 40 : au plus 40 nouveaux tokens sont générés.
• temperature = 0.8 : les probabilités sont un peu plus concentrées que pour T = 1.
• top_k = 20 : seuls les 20 tokens les mieux classés peuvent être sélectionnés à chaque étape.

Ces valeurs sont un point de départ expérimental, pas une configuration universellement optimale. Le résultat dépend du modèle, du prompt et des données d'entraînement.

6. Résumé final

Logits : scores bruts calculés par le modèle.
Température : règle la concentration de la distribution après Softmax, en divisant les logits avant Softmax.
top_k : conserve uniquement les k meilleurs candidats et exclut les autres.
Échantillonnage : choisit ensuite un token à partir des probabilités obtenues.

Ordre typique du décodage : logits → division par la température → filtrage top_k → Softmax → échantillonnage du prochain token → ajout au contexte → répétition.

Les paramètres de décodage contrôlent la génération ; ils ne remplacent ni l'entraînement ni l'évaluation sur un ensemble de validation distinct.